

Lecture Notes: Graph Algorithms (Part I)

For BSDS Students, Semester 3

November 26, 2025

1 Graph Representation

Definition 1.1 (Graph). A graph G is a pair (V, E) :

- $V \rightarrow$ set of vertices (nodes)
- $E \rightarrow$ set of edges (connections between vertices)

1.1 Adjacency Matrix

Structure: For a graph with $n = |V|$ vertices (numbered 1 to n), use an $n \times n$ matrix A .

$$A[i][j] = \begin{cases} 1 & \text{if there is an edge from } i \rightarrow j \\ \text{weight}(i, j) & \text{for weighted graphs (store weight instead of 1)} \\ 0 & \text{if there is no edge} \end{cases}$$

For **undirected graphs**, the matrix is **symmetric**: $A[i][j] = A[j][i]$.

Example 1.1. Graph: Vertices: $A = 1, B = 2, C = 3, D = 4$. Edges: $(A, B), (A, C), (B, D), (C, D)$. The adjacency matrix is:

	A	B	C	D
A	0	1	1	0
B	1	0	0	1
C	1	0	0	1
D	0	1	1	0

Table 1: Adjacency Matrix Properties

Aspect	Adjacency Matrix
Space	$O(V ^2)$
Check if (u, v) exists	$O(1)$
Traverse all neighbors of v	$O(V)$
Best for	Dense graphs
Direction support	Easily handled
Weight support	Yes (store $w(u, v)$)

Advantages: Simple, direct lookup for edges. Easy to implement for dense graphs. Works well with matrix-based algorithms (e.g., Floyd–Warshall). **Disadvantages:** High space cost for sparse graphs. Inefficient neighbor iteration if most entries are zero.

1.2 Adjacency List

Structure: Store for each vertex a list of its adjacent vertices (neighbors). Implemented using linked lists, dynamic arrays, or vectors.

Example 1.2. $A: B, C$

$B: A, D$

$C: A, D$

$D: B, C$

Table 2: Adjacency List Properties

Aspect	Adjacency List
Space	$O(V + E)$
Check if (u, v) exists	$O(\deg(u))$
Traverse neighbors of v	$O(\deg(v))$
Best for	Sparse graphs
Weighted graphs	Store (neighbor, weight) pairs

Advantages: Much more space-efficient for sparse graphs. Fast to iterate over all outgoing edges from a vertex. Flexible to add/remove edges. **Disadvantages:** Slower to check existence of a specific edge. Slightly more complex to implement.

1.3 Edge List

Structure: Simply store all edges as a list (array) of pairs (u, v) (or triples (u, v, w) for weighted graphs).

Edges = $[(A, B), (A, C), (B, D), (C, D)]$

Weighted Edges = $[(A, B, 4), (A, C, 2), (B, D, 5), (C, D, 1)]$

Table 3: Edge List Properties

Aspect	Edge List
Space	$O(E)$
Check if (u, v) exists	$O(E)$
Traverse neighbors	$O(E)$ (need to filter)
Best for	Algorithms that only need edge iteration (e.g. Kruskal's)

1.4 Adjacency Set (Variant)

Uses a dictionary or hash set for each vertex's neighbors. Provides $O(1)$ average lookup for edge existence, combining adjacency list space efficiency with fast edge checks.

1.5 Comparison Summary

Which One to Use?

- **Dense graph (many edges):** Adjacency Matrix
- **Sparse graph:** Adjacency List
- **When edge sorting matters:** Edge List
- **When frequent edge existence checks matter:** Adjacency Set / Matrix

Table 4: Comparison of Graph Representations

Representation	Space	Edge lookup	Neighbor traversal	Best for
Adjacency Matrix	$O(V ^2)$	$O(1)$	$O(V)$	Dense graphs
Adjacency List	$O(V + E)$	$O(\deg(v))$	$O(\deg(v))$	Sparse graphs
Edge List	$O(E)$	$O(E)$	$O(E)$	Edge-based algorithms
Adjacency Set	$O(V + E)$	$O(1)$ avg	$O(\deg(v))$	Medium graphs

2 Graph Traversal

Definition 2.1 (Graph Search). Graph search means visiting all vertices reachable from a given source.

2.1 Depth-First Search (DFS)

DFS explores as far as possible along each branch before backtracking. Uses a **stack** (explicit or via recursion).

2.1.1 Version 1: Template for Problem Solving

This version shows how to insert custom operations (**preWORK** and **postWORK**) into the traversal logic.

```

1 Algorithm DFS (G, v);
2 Input: G = (V, E) (an undirected graph), and v (a vertex of G)
3 Output: depends on the application
4 begin
5     mark v;
6     perform preWORK on v;    //preWORK depends on the application of DFS
7     for all edges (v, w) do
8         if w is unmarked then
9             DFS(G, w)
10            perform postWORK for (v, w)    //postWORK depends on the application,
11                                           sometimes postWORK is done only on edges leading to unmarked vertices
11 end

```

Listing 1: DFS Algorithm (Template Version)

```

1 Algorithm DFS-Numbering (G, v);
2 Input: G = (V, E) (an undirected graph), and v (a vertex of G)
3 Output: for every vertex v, v.DFS is set to the DFS number of v
4 begin
5     initially DFS_Number := 1
6     use DFS with following preWORK:
7         v.DFS = DFS_Number
8         DFS_Number = DFS_Number + 1
9 end

```

Listing 2: DFS Algorithm (Template Version)

```

1 Algorithm DFS-Tree (G, v);
2 Input: G = (V, E) (an undirected graph), and v (a vertex of G)
3 Output: T (a DFS tree of G, initially T is empty)
4 begin
5     use DFS with following postWORK:
6         if w was unmarked then add the edge (v,w) to T
7 end

```

Listing 3: DFS Algorithm (Template Version)

Edges of a directed graph can be classified into tree edge, back edge, forward edge, and cross edge with respect to a DFS Tree.

Lemma 2.1. *If G is connected, then all its vertices will be marked by algorithm DFS, and all its edges will be looked at at least once during the execution of the algorithm.*

Lemma 2.2. *Let G be a connected and undirected graph, and let $T = (V, F)$ be A DFS tree of G constructed by algorithm DFS-Tree. Every edge $e \in E$ either belongs to T or connects two vertices of G , one of which is an ancestor of the other in T .*

2.1.2 Version 2: Pseudocode with Timestamps and Colors

This version tracks the discovery and finish times, which are crucial for advanced applications like topological sort and finding connected components.

```

1 DFS(G):
2     for each vertex v in V[G]:
3         color[v] = WHITE
4         parent[v] = NIL
5     time = 0
6     for each vertex v in V[G]:
7         if color[v] == WHITE:
8             DFS-VISIT(G, v)
9
10 DFS-VISIT(G, u):
11     color[u] = GRAY           // u discovered (processing)
12     time = time + 1
13     discovery[u] = time
14     for each vertex v in Adj[u]:
15         if color[v] == WHITE:
16             parent[v] = u
17             DFS-VISIT(G, v)
18     color[u] = BLACK         // u finished (fully explored)
19     time = time + 1
20     finish[u] = time

```

Listing 4: DFS Algorithm (Recursive, Time/Color Tracking)

Table 5: DFS Complexity

Aspect	Complexity
Time	$O(V + E)$
Space	$O(V)$ (due to recursion/stack)

Lemma 2.3. *Let $G = (V, E)$ be a directed graph, and let $T = (V, F)$ be a DFS tree of G . If (v, w) is an edge of E such that $v.DFS_{Number} < w.DFS_{Number}$, then w is a descendant of v in tree T .*

Lemma 2.4. *Let $G = (V, E)$ be a directed graph, and let T be a DFS tree of G . Then G contains a directed cycle if and only if G contains a back edge relative to T .*

2.2 Breadth-First Search (BFS)

BFS explores neighbors level by level (by path length from the source). Uses a **Queue** (First In First Out, FIFO).

2.2.1 Version 1: Template for Problem Solving

This iterative version uses a queue and explicitly builds a traversal tree T .

```

1 Algorithm BFS (G, v);
2 Input: G=(V, E) (an undirected graph), and v (a vertex of G)
3 Output: depends on the application
4 begin
5     mark v;
6     put v in a queue {First In First Out}
7     while the queue is not empty do
8         remove the first vertex w from the queue;
9         perform preWORK on w;
10        for all edges (w, x) such that x is unmarked do
11            mark x;
12            add (w, x) to the tree T
13            put x in the queue
14 end

```

Listing 5: BFS Algorithm (Template Version)

Lemma 2.5. *If an edge (u, w) belongs to a BFS tree, such that u is a parent of w , then u has the minimal BFS number among vertices with edges leading to w .*

Lemma 2.6. *For each vertex w , the path from the root to w in T is a shortest path from root to w in G .*

Lemma 2.7. *If (v, w) is an edge in E that does not belong to T , then it connects two vertices whose level numbers differ by at most 1.*

2.2.2 Version 2: Pseudocode with Distance and Parent Tracking

This version explicitly calculates the shortest path distance (number of edges) and the path structure (parent array).

```

1 BFS(G, s):
2     for each vertex v in V[G]:
3         color[v] = WHITE           // unvisited
4         distance[v] = inf
5         parent[v] = NIL
6     color[s] = GRAY                // discovered
7     distance[s] = 0
8     parent[s] = NIL
9     Q = empty queue
10    ENQUEUE(Q, s)
11
12    while Q is not empty:
13        u = DEQUEUE(Q)
14        for each vertex v in Adj[u]:
15            if color[v] == WHITE:
16                color[v] = GRAY
17                distance[v] = distance[u] + 1
18                parent[v] = u
19                ENQUEUE(Q, v)
20    color[u] = BLACK                // fully explored

```

Listing 6: BFS Algorithm (Distance/Parent Tracking)

2.3 BFS vs DFS Comparison

Table 6: BFS Complexity

Aspect	Complexity
Time	$O(V + E)$
Space	$O(V)$ (for auxiliary structures and queue)

Table 7: Traversal Comparison

Feature	BFS	DFS
Data structure	Queue	Stack / Recursion
Search pattern	Level-wise	Depth-wise
Finds shortest path (unweighted)	Yes	No
Time complexity	$O(V + E)$	$O(V + E)$
Applications	Shortest path, web crawling	Topological sort, cycle detection

3 Shortest Paths and Ordering

3.1 Dijkstra's Algorithm (Single Source Shortest Paths)

Dijkstra's Algorithm (Shortest First Search, SFS) is a **greedy** algorithm for finding the shortest path from a single source in a weighted graph with **non-negative edge weights**.

Data Structures Used: Adjacency list, Priority Queue (Min-heap), distance array `dist[]`.

```

1 ShortestFirstSearch(Graph, source):
2   // Input: Graph = (V, E) with non-negative edge weights
3   for each vertex v in V:
4       dist[v] <- inf
5       parent[v] <- NIL
6   dist[source] <- 0
7   PQ <- empty min-priority queue (keyed by dist[])
8   PQ.insert(source, 0)
9
10  while PQ is not empty:
11      (u, du) <- PQ.extract_min()           // O(log |V|)
12      for each edge (u, v) with weight w in Adj[u]:
13          if dist[v] > dist[u] + w:
14              dist[v] <- dist[u] + w
15              parent[v] <- u
16              if v not in PQ:
17                  PQ.insert(v, dist[v])     // O(log |V|)
18              else:
19                  PQ.decrease_key(v, dist[v]) // O(log |V|)

```

Listing 7: Dijkstra's Algorithm

Table 8: Dijkstra's Algorithm Time Complexity

Implementation	Time Complexity
Adjacency List + Min-Heap (Binary Heap)	$O((V + E) \log V)$
Adjacency List + Fibonacci Heap	$O(E + V \log V)$

Remarks: Works only for non-negative edge weights. For negative weights, use Bellman-Ford or Johnson's algorithm. BFS is a special case of SFS where all edge weights are 1.

3.2 Topological Sort

Topological sort produces a linear ordering of the vertices of a **Directed Acyclic Graph (DAG)** such that for every directed edge $u \rightarrow v$, u appears before v .

```

1 Algorithm Topological_Sort (G, v);
2 Input: G = (V, E) (a directed acyclic graph)
3 Output: The Label field indicates a topological sorting of G
4 begin
5     initialize v.Indegree for all vertices (can be done using DFS)
6     G_label = 0;
7     for i= 1 to n do
8         if v_i.Indegree = 0 then put v_i in Queue;
9     repeat
10        remove vertex v from Queue;
11        G_label = G_label + 1;
12        v_label = G_label;
13        for all edges (v,w) do
14            w.Indegree = w.Indegree - 1;
15            if w.Indegree = 0 then put w in Queue;
16    until Queue is empty
17 end

```

Listing 8: Topological Sort

3.2.1 Algorithm A — Kahn's algorithm (Queue / Indegree Method)

Idea: Repeatedly remove vertices with indegree = 0.

```

1 KAHN_TOPOLOGICAL_SORT(G):
2     // Compute initial indegrees
3     for each v in V: indeg[v] <- 0
4     for each u->v in E: indeg[v] <- indeg[v] + 1
5
6     Q = empty queue
7     for each v in V:
8         if indeg[v] == 0: ENQUEUE(Q, v)
9
10    topo = empty list
11    while Q not empty:
12        u <- DEQUEUE(Q)
13        append u to topo
14        for each v in Adj[u]:
15            indeg[v] <- indeg[v] - 1
16            if indeg[v] == 0:
17                ENQUEUE(Q, v)
18
19    if length(topo) < |V|:
20        return "Graph has a cycle"
21    else:
22        return topo

```

Listing 9: Kahn's Topological Sort

Complexity: $O(|V| + |E|)$.

3.2.2 Algorithm B — DFS-based Topological Sort (Finish-Time Method)

Idea: Run DFS and record vertices in the reverse of their finish-time order. Cycle detection is done by finding a back edge (GRAY neighbor).

```

1 DFS_TOPOLOGICAL_SORT(G):
2     for each v in V: color[v] <- WHITE

```

```

3     topo = empty list
4     for each v in V:
5         if color[v] == WHITE:
6             if DFS-VISIT(v) == "cycle":
7                 return "Graph has a cycle"
8     return reverse(topo) // reverse of finish order is topo order
9
10 DFS-VISIT(u):
11     color[u] <- GRAY
12     for each v in Adj[u]:
13         if color[v] == WHITE:
14             if DFS-VISIT(v) == "cycle": return "cycle"
15         else if color[v] == GRAY:
16             return "cycle" // Back edge found -> directed cycle
17     color[u] <- BLACK
18     append u to topo // record finish order
19     return "ok"

```

Listing 10: DFS-based Topological Sort

Complexity: $O(|V| + |E|)$.

4 Connected Components

4.1 Connected Components (Undirected Graphs)

A connected component is a maximal set of vertices where every pair is connected by a path.

Algorithms: Either DFS or BFS can find connected components in $O(|V| + |E|)$ time (using an adjacency list).

```

1 ConnectedComponents_BFS(Graph):
2     for each vertex v in V: visited[v] = false
3     component_id = 0
4     for each vertex s in V:
5         if not visited[s]:
6             component_id = component_id + 1
7             queue = empty queue; visited[s] = true; comp[s] = component_id
8             enqueue(queue, s)
9             while queue not empty:
10                u = dequeue(queue)
11                for each w in Adj[u]:
12                    if not visited[w]:
13                        visited[w] = true
14                        comp[w] = component_id
15                        enqueue(queue, w)

```

Listing 11: Finding Connected Components (BFS-based)

Complexity: $O(|V| + |E|)$.

4.2 Strongly Connected Components (SCCs) (Directed Graphs)

An SCC is a maximal set of vertices in a directed graph such that every vertex is reachable from every other vertex in the set.

4.2.1 Kosaraju's Algorithm (Two-Pass DFS)

1. **First Pass (DFS on G):** Compute the finish order and store it in a stack.
2. **Transpose:** Compute G^T (reverse all edges).
3. **Second Pass (DFS on G^T):** Process vertices in decreasing finish time order (pop from stack). Each DFS tree in this pass is an SCC.

Complexity: $O(|V| + |E|)$.

4.2.2 Tarjan's Algorithm (Single-Pass DFS, Low-Link)

Uses a single DFS, tracking $\text{disc}[v]$ and $\text{low}[v]$. An SCC is detected when a vertex v 's low value equals its disc value, indicating v is the root of the SCC.

```

1 strongconnect(v):
2     index_of[v] = index
3     lowlink[v] = index
4     index = index + 1
5     push(stack, v); onstack[v] = true
6
7     for each w in Adj[v]:
8         if index_of[w] == -1:
9             strongconnect(w)
10            lowlink[v] = min(lowlink[v], lowlink[w])
11        else if onstack[w]:
12            lowlink[v] = min(lowlink[v], index_of[w]) // Back edge
13
14    // If v is a root node, pop the stack and generate an SCC
15    if lowlink[v] == index_of[v]:
16        scc = empty list
17        repeat
18            w = pop(stack)
19            onstack[w] = false
20            add w to scc
21        until w == v
22        output scc

```

Listing 12: Tarjan's Algorithm Core Logic (strongconnect)

Complexity: $O(|V| + |E|)$.

4.3 Bi-Connected Components (BCCs) (Undirected Graphs)

A BCC is a maximal subgraph of an undirected graph where removing any single vertex does not disconnect the subgraph.

Tarjan's Algorithm for BCCs: Uses a DFS, $\text{disc}[v]$, $\text{low}[v]$, and an edge stack S . BCCs are identified when an articulation point condition is met, and edges are popped from S to form the component.

Articulation Point Conditions (Vertex u):

- u is the **root** of DFS tree and has ≥ 2 children.
- u is **not root**, and $\exists$ child v such that $\text{low}[v] \geq \text{disc}[u]$.

Complexity: $O(|V| + |E|)$.

5 Problem Statement

Given a **Directed Acyclic Graph (DAG)** with edge weights (which may be positive, zero, or negative but no cycles), we want to find the **shortest path distance** from a single source vertex s to all other vertices in the graph.

6 Key Idea

Because the graph is acyclic, we can use **Topological Sorting** to linearize the vertices such that all edges go from earlier to later in the order. Then we can relax each edge once in that topological order.

This approach avoids the need for Dijkstra's or Bellman-Ford's multiple relaxations and works efficiently in $\mathcal{O}(V + E)$ time.

7 Algorithm Steps

Step 1: Topological Sort

Perform a topological sort of the DAG using DFS or in-degree counting (Kahn's algorithm). This gives an order in which all edges go from left to right.

Step 2: Initialize Distances

Let $\text{dist}[v]$ denote the shortest distance from source s to vertex v . Initialize all distances as ∞ , except $\text{dist}[s] = 0$.

Step 3: Relax Edges in Topological Order

Traverse the vertices in topological order. For each vertex u , for every edge (u, v) with weight $w(u, v)$:

if $\text{dist}[v] > \text{dist}[u] + w(u, v)$ then

$\text{dist}[v] = \text{dist}[u] + w(u, v)$

and update $\text{parent}[v] = u$ (if you also want to reconstruct paths).

Step 4: (Optional) Reconstruct Paths

You can reconstruct each shortest path from s to v using the $\text{parent}[]$ array.

8 Pseudocode for a combined approach

```

1 Algorithm Acyclic_Shortest_Path (G, v);
2 Input: G = (V, E) (a directed acyclic graph), v (a vertex of G)
3 Output: For every vertex w, w.SP is the length of the shortest path from v to w
4
5 begin
6   for all vertices w do
7     w.SP = \infty
8   Initialize v.Indegree for all vertices
9   for i= 1 to n do
10    if v_i.Indegree = 0 then put v_i in Queue;
11  v.SP = 0;
12  repeat
13    remove vertex w from Queue
14    for all edges (w,z) do
15      if w.SP + length(w,z) < z.SP then
16        z.SP = w.SP + length(w,z)
17        z.Indegree = z.Indegree - 1;
18        if z.Indegree = 0 then put z in Queue
19  until Queue is empty
20 end

```

Listing 13: Acyclic-Shortest-Path

```

1 Algorithm SSSP (G, v);
2 Input: G = (V, E) (a weighted directed graph), v (the source vertex)
3 Output: For every vertex w, w.SP is the length of the shortest path from v to w
4 Assumption: All weights are non-negative.
5
6 begin

```

```

7   for all vertices w do
8       w.mark = false
9       w.SP = \infty
10  v.SP = 0;
11  while there exists an unmarked vertex do
12      let w be an unmarked vertex such that w.SP is minimal
13      w.mark = true
14      for all edges (w,z) such that z is unmarked do
15          if w.SP + length(w,z) < z.SP then
16              z.SP = w.SP + length(w,z)
17  end

```

Listing 14: Single-Source-Shortest-Paths

9 Problem Statement

Given a connected, undirected, weighted graph $G = (V, E)$, the goal is to find a **Minimum Cost Spanning Tree (MCST)** — a subset of edges $T \subseteq E$ that connects all vertices with the minimum possible total edge weight and contains no cycles.

10 Key Idea

Kruskal's algorithm is a greedy algorithm that builds the spanning tree by always choosing the smallest-weight edge that does not form a cycle with the edges already included.

To efficiently check for cycles, it uses the **Union-Find (Disjoint Set Union, DSU)** data structure, which supports: - **Find(x)**: Returns the representative (root) of the set containing vertex x . - **Union(x, y)**: Merges the sets containing x and y .

With *union by rank* and *path compression*, these operations run in almost constant amortized time.

11 Algorithm Steps

Step 1: Sort All Edges

Sort all edges in non-decreasing order of their weights.

Step 2: Initialize Disjoint Sets

Create a disjoint set for each vertex using the Union-Find structure.

Step 3: Build the MST

Iterate through the sorted edges (u, v) :

- If $\text{Find}(u) \neq \text{Find}(v)$, then including this edge does not form a cycle.
- Add (u, v) to the MST and perform $\text{Union}(u, v)$.

Step 4: Stop Condition

The process continues until the MST contains exactly $|V| - 1$ edges.

12 Pseudocode

Algorithm $\text{Kruskal_MST}(G)$:

Input: Undirected weighted graph $G = (V, E)$

Output: Set of edges forming a Minimum Spanning Tree (MST)

```

1. A =                                // Set to store MST edges
2. for each vertex v in V:
    MakeSet(v)                        // Initialize disjoint sets

3. sort edges E in non-decreasing order by weight w(u, v)

4. for each edge (u, v) in sorted order:
    if Find(u) != Find(v):
        A = A ∪ {(u, v)}
        Union(u, v)

5. return A

```

Subprocedure: Union-Find Operations

```

MakeSet(x):
    parent[x] = x
    rank[x] = 0

```

```

Find(x):
    if parent[x] != x:
        parent[x] = Find(parent[x])    // Path compression
    return parent[x]

```

```

Union(x, y):
    rootX = Find(x)
    rootY = Find(y)
    if rootX == rootY:
        return
    if rank[rootX] < rank[rootY]:
        parent[rootX] = rootY
    else if rank[rootX] > rank[rootY]:
        parent[rootY] = rootX
    else:
        parent[rootY] = rootX
        rank[rootX] = rank[rootX] + 1

```

13 Complexity Analysis

Operation	Time Complexity
Sorting of edges	$O(E \log E)$
Find/Union operations (amortized)	$O(\alpha(V))$ each
Overall time complexity	$O(E \log E)$
Space complexity	$O(V)$

Here, $\alpha(V)$ is the inverse Ackermann function, which grows extremely slowly, and is effectively constant for all practical values of V .

14 Example

Consider a graph with 4 vertices and the following weighted edges:

$$E = \{(A, B, 1), (B, C, 2), (A, C, 3), (C, D, 4), (B, D, 5)\}$$

After sorting edges by weight and applying Kruskal's algorithm:

MST edges: $\{(A, B, 1), (B, C, 2), (C, D, 4)\}$

Total weight: $1 + 2 + 4 = 7$

15 Proof of Correctness of Kruskal's Algorithm

Theorem

Kruskal's Algorithm correctly produces a Minimum Cost Spanning Tree (MCST) of a connected, undirected, weighted graph $G = (V, E)$.

Proof Outline

We prove correctness using two main properties of spanning trees:

1. The **Cycle Property**
2. The **Cut Property**

1. Greedy Choice Based on the Cut Property

Let S be any subset of vertices of V that does not include all vertices, and let (u, v) be the edge of minimum weight that connects a vertex in S to a vertex in $V \setminus S$.

Cut Property: The minimum-weight edge (u, v) that crosses any cut $(S, V \setminus S)$ is part of *some* minimum spanning tree of G .

Proof: Suppose not. Let T be an MST that does not include (u, v) . Since T is a spanning tree, adding (u, v) to T forms a cycle. Within this cycle, there must exist another edge (x, y) that crosses the same cut $(S, V \setminus S)$. By definition, $w(u, v) \leq w(x, y)$ since (u, v) is the lightest edge across the cut.

Now remove (x, y) from $T \cup \{(u, v)\}$. The resulting graph $T' = T \cup \{(u, v)\} - \{(x, y)\}$ is still a spanning tree (since it has $|V| - 1$ edges and remains connected), and its total weight satisfies:

$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$$

Hence, T' is also a minimum spanning tree, and it includes (u, v) , which contradicts our assumption. Thus, the cut property holds.

2. Preservation of Acyclic Structure

Kruskal's algorithm always adds an edge (u, v) only if it connects two different components. This ensures that no cycles are formed, since $\text{Find}(u) \neq \text{Find}(v)$ before the edge is added. The Union-Find structure maintains this invariant efficiently.

Therefore, every step of the algorithm adds an edge that:

- preserves acyclicity, and
- satisfies the cut property (it is a safe edge to add).

3. Termination and Minimality

After processing all edges (in increasing order of weight), Kruskal's algorithm stops when exactly $|V| - 1$ edges have been added. At this point:

- The selected edges form a connected, acyclic graph (a spanning tree).
- By the cut property, every edge added is part of some MST.
- Therefore, the resulting tree is itself an MST.

Conclusion

Since Kruskal's algorithm repeatedly adds the lightest edge that connects two distinct components (a safe edge) and never forms a cycle, it follows by the cut property that:

The final spanning tree produced is a Minimum Cost Spanning Tree.

□

16 Prim's Algorithm for Minimum Cost Spanning Tree (MCST)

Problem Statement

Given a connected, undirected, weighted graph $G = (V, E)$, the goal is to find a **Minimum Cost Spanning Tree (MCST)** — a subset of edges $T \subseteq E$ that connects all vertices with the minimum possible total edge weight and contains no cycles.

Key Idea

Prim's algorithm is a **greedy** algorithm that builds the spanning tree incrementally. It starts with an arbitrary source vertex and repeatedly adds the minimum-weight edge that connects a vertex already in the tree to a vertex outside it.

The algorithm maintains two disjoint sets of vertices:

- S : the set of vertices included in the MST (the “tree” so far)
- $V \setminus S$: the set of vertices not yet included

At each step, Prim's algorithm adds the lightest edge (u, v) where $u \in S$ and $v \in V \setminus S$.

Algorithm Steps

1. Initialize all vertices with key value ∞ , except the source vertex s with key value 0.
2. Maintain a priority queue (min-heap) that stores all vertices keyed by their minimum edge weight to S .
3. Repeatedly extract the vertex u with the smallest key value and add it to S .
4. For every vertex v adjacent to u , if $v \notin S$ and $w(u, v) < \text{key}[v]$, then update:

$$\text{key}[v] = w(u, v), \quad \text{parent}[v] = u$$

5. Continue until all vertices are included in S .

Pseudocode

Algorithm Prim_MST(G, w, s):

Input: Undirected weighted graph $G = (V, E)$
 weight function $w(u, v)$
 source vertex s

Output: Set of edges forming a Minimum Spanning Tree (MST)

1. for each vertex v in V :
 $\text{key}[v] = \infty$
 $\text{parent}[v] = \text{NIL}$
2. $\text{key}[s] = 0$
3. $Q = \text{all vertices in } V$ // Min-priority queue (by key value)

```

4. S =                                     // Set of vertices in MST

5. while Q :
    u = Extract-Min(Q)
    S = S ∪ {u}

    for each vertex v in Adj[u]:
        if v ∉ Q and w(u, v) < key[v]:
            parent[v] = u
            key[v] = w(u, v)
            Decrease-Key(Q, v, key[v])

6. return parent[]

```

Proof of Correctness

Theorem: Prim's Algorithm correctly produces a Minimum Cost Spanning Tree (MCST) of a connected, undirected, weighted graph $G = (V, E)$.

Proof:

Prim's algorithm grows a single tree by repeatedly adding the minimum-weight edge that connects the current tree S to a vertex outside S . We show that each chosen edge is *safe* using the **Cut Property**.

Cut Property: Let $S \subset V$ be the set of vertices already included in the tree, and let (u, v) be the edge of minimum weight connecting S and $V \setminus S$. Then (u, v) is part of some MST of G .

Proof of Cut Property: Suppose (u, v) is not in a particular MST T . Adding (u, v) to T creates a cycle. In this cycle, there must exist another edge (x, y) crossing the same cut $(S, V \setminus S)$. Since (u, v) has minimum weight among all such edges,

$$w(u, v) \leq w(x, y)$$

Now, remove (x, y) from $T \cup \{(u, v)\}$ to obtain $T' = T \cup \{(u, v)\} - \{(x, y)\}$. T' remains connected, acyclic, and spans all vertices, and

$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T)$$

Thus T' is also an MST and contains (u, v) . Therefore, (u, v) is safe to add.

Inductive Argument: Initially, $S = \{s\}$ and the invariant trivially holds. At each iteration, the algorithm adds one safe edge by the cut property, preserving the invariant that S induces a minimum-cost tree on its vertices.

Termination: When all vertices have been added ($S = V$), the set of edges forms a spanning tree with minimum total cost. Thus, Prim's algorithm produces a valid MST.

□

Complexity Analysis

Implementation Variant	Time Complexity
Using adjacency matrix and no heap	$O(V^2)$
Using binary heap and adjacency list	$O(E \log V)$
Using Fibonacci heap	$O(E + V \log V)$
Space Complexity	$O(V + E)$

Remarks

- Kruskal's algorithm sorts edges globally; Prim's algorithm grows a tree locally.
- Both are greedy algorithms and rely on the **cut property**.
- For dense graphs, Prim's algorithm (with $O(V^2)$ implementation) is efficient; for sparse graphs, the binary heap version ($O(E \log V)$) is preferable.

17 Perceptron Learning Algorithm

Pseudocode

[H] Perceptron_Train($\{(x_i, y_i)\}_{i=1}^m$) [1] Training set $\{(x_i, y_i)\}_{i=1}^m$, where $x_i \in \mathbb{R}^d$, $y_i \in \{-1, +1\}$ Weight vector w and bias b that separate data (if separable) $w \leftarrow \mathbf{0} \in \mathbb{R}^d$ $b \leftarrow 0$ **error** $\leftarrow$ **false** $i \leftarrow 1$ **to** m $y_i(w \cdot x_i + b) \leq 0$ misclassified or on boundary $w \leftarrow w + y_i x_i$ $b \leftarrow b + y_i$ **error** $\leftarrow$ **true** **error** = **false** w, b

Note: the loop terminates when a full pass over the training set produces no updates (i.e., all points correctly classified). For large data, one may instead run for a fixed number of epochs.

Complexity analysis

We derive two things: (A) the *mistake bound* (number of updates), and (B) how runtime estimates follow from that bound.

Notation and assumptions

- Let $(a_1, \dots, a_m) = (x_1, y_1), \dots, (x_m, y_m)$ be the training data.
- Assume data are *linearly separable*: there exists a vector w^* with $\|w^*\| = 1$ and a margin $\gamma > 0$ such that

$$y_i(w^* \cdot x_i) \geq \gamma \quad \text{for all } i.$$

- Let $R = \max_i \|x_i\|$ (bound on input norms).
- Let t denote the total number of *updates* the perceptron performs before convergence.

(A) Mistake bound **Lemma 1 (progress along w^*).** After each update using example (x_i, y_i) ,

$$w_{t+1} \cdot w^* \geq w_t \cdot w^* + \gamma.$$

Proof. Update: $w_{t+1} = w_t + y_i x_i$. Hence

$$w_{t+1} \cdot w^* = w_t \cdot w^* + y_i(x_i \cdot w^*) \geq w_t \cdot w^* + \gamma,$$

by the margin assumption. $\square$

Lemma 2 (bound on squared norm). After t updates,

$$\|w_t\|^2 \leq tR^2.$$

Proof. For an update $w_{t+1} = w_t + y_i x_i$,

$$\|w_{t+1}\|^2 = \|w_t\|^2 + 2y_i(w_t \cdot x_i) + \|x_i\|^2.$$

Because update occurs only when $y_i(w_t \cdot x_i) \leq 0$, we have

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + \|x_i\|^2 \leq \|w_t\|^2 + R^2.$$

Inducting over t updates yields $\|w_t\|^2 \leq tR^2$. $\square$

Combine lemmas to get mistake bound. From Lemma 1,

$$w_t \cdot w^* \geq t\gamma.$$

By Cauchy–Schwarz,

$$w_t \cdot w^* \leq \|w_t\| \|w^*\| = \|w_t\|.$$

Therefore $t\gamma \leq \|w_t\|$. Using Lemma 2,

$$t\gamma \leq \sqrt{t}R \quad \Rightarrow \quad t \leq \frac{R^2}{\gamma^2}.$$

This is the standard perceptron *mistake bound*.

(B) From mistake bound to runtime estimates Cost per arithmetic operation:

- Evaluating $y_i(w \cdot x_i + b)$ costs $O(d)$ time (a dot product of dimension d plus one addition).
- An update $w \leftarrow w + y_i x_i$ costs $O(d)$ time (adding scaled vector).

Counting updates: By the mistake bound, the number of updates t satisfies

$$t \leq \frac{R^2}{\gamma^2}.$$

Thus the *work spent on updates* is

$$O(t \cdot d) = O\left(d \cdot \frac{R^2}{\gamma^2}\right).$$

Additional work for checks (dot-product evaluations): The algorithm scans examples and for each example computes $y_i(w \cdot x_i + b)$ to decide whether to update. Each such check costs $O(d)$. Two standard runtime perspectives follow:

Optimistic / updates-dominated view: Count only the arithmetic work tied to updates. If one assumes the implementation jumps quickly from one mistake to the next (or we only charge cost to updates), the dominant cost is $O(d \cdot (R^2/\gamma^2))$.

Pessimistic / worst-case examination view: In the worst case, between two updates the algorithm might examine up to all m examples once. A crude upper bound on the total number of example examinations is $O(m \cdot t)$, giving worst-case time

$$O\left(md \cdot \frac{R^2}{\gamma^2}\right).$$

This bound is loose but valid: each update may be preceded by up to m checks in the worst scheduling/order of data.

Practical summary:

- The theoretically meaningful bound is the *mistake bound* $t \leq R^2/\gamma^2$; each update costs $O(d)$, so updates contribute $O(dR^2/\gamma^2)$ arithmetic operations.
- If you measure running time in terms of passes over the data, each pass costs $O(md)$ and the number of passes until convergence is at most $O(R^2/(\gamma^2 m))$ in an averaged sense; conservatively use $O(md \cdot R^2/\gamma^2)$ as a worst-case bound.
- Space complexity remains $O(d)$ for the weight vector (plus $O(1)$ for bias).

Implementation notes

- **Shuffling:** Randomizing the order of examples each epoch often helps convergence in practice.
- **Learning rate:** The basic perceptron uses step size 1; a learning rate $\eta > 0$ can be used: $w \leftarrow w + \eta y_i x_i$, scaling the bounds accordingly.
- **Numerical stability:** Normalizing inputs (e.g., scale so $\|x_i\| \leq 1$) makes R small and tightens the bound.
- **Early stopping / epoch limit:** For non-separable data use a maximum number of epochs or switch to variants (pocket algorithm, averaged perceptron).